

Tutoriel

Utilisation de Java KeyStore (Keystore) pour la gestion des clés et certificats

1. Introduction

Le **Java KeyStore (JKS)** est un composant essentiel de la sécurité en Java. Il permet de stocker de manière sécurisée :

- **Clés privées**
- **Clés publiques**
- **Certificats numériques**
- **Chaînes de certificats**

Il est utilisé dans :

- Applications **web** (HTTPS, SSL/TLS)
 - Applications **desktop** (signature, chiffrement, authentification)
-

2. Concepts fondamentaux

2.1 Keystore vs Truststore

- **Keystore** : contient les **clés privées + certificats associés**
- **Truststore** : contient les **certificats de confiance (CA)**

2.2 Types de Keystore

- **JKS** (Java KeyStore - ancien format)
- **PKCS12** (standard recommandé aujourd'hui)

Depuis Java 9, **PKCS12 est le format par défaut**

3. L'outil keytool

Java fournit un outil en ligne de commande appelé :

keytool

Il permet de :

- Créer des clés
 - Générer des certificats
 - Importer/exporter des certificats
 - Gérer les entrées du keystore
-

4. Création d'un Keystore et d'une paire de clés

4.1 Générer une paire de clés

```
keytool -genkeypair \  
-alias monalias \  
-keyalg RSA \  
-keysize 2048 \  
-keystore monkeystore.p12 \  
-storetype PKCS12 \  
-validity 365
```

Explication :

- -alias : nom de la clé
 - -keyalg : algorithme (RSA recommandé)
 - -keysize : taille de la clé
 - -keystore : fichier de stockage
 - -validity : durée de validité (jours)
-

5. Gestion des certificats

5.1 Exporter un certificat

```
keytool -exportcert \  
-alias monalias \  
-keystore monkeystore.p12 \  
-file certificat.cer
```

5.2 Importer un certificat

```
keytool -importcert \  
-alias moncertificat \  
-file certificat.cer \  
-keystore monkeystore.p12
```

5.3 Lister le contenu du Key store

```
keytool -list -keystore monkeystore.p12
```

5.4 Supprimer une entrée

```
keytool -delete  
-alias monalias  
-keystore monkeystore.p12
```

6. Utilisation dans une application Java Desktop

6.1 Charger un Key store en Java

```
KeyStore keyStore = KeyStore.getInstance("PKCS12");  
FileInputStream fis = new FileInputStream("monkeystore.p12");  
keyStore.load(fis, "motdepasse".toCharArray());
```

6.2 Récupérer une clé privée

```
Key key = keyStore.getKey("monalias", "motdepasse".toCharArray());
```

```
if (key instanceof PrivateKey) {  
    PrivateKey privateKey = (PrivateKey) key;  
}
```

6.3 Récupérer le certificat

```
Certificate cert = keyStore.getCertificate("monalias");  
PublicKey publicKey = cert.getPublicKey();
```

7. Utilisation dans une application Web (HTTPS)

7.1 Configuration SSL (ex : Spring Boot)

Dans application.properties :

```
server.port=8443  
server.ssl.key-store=classpath:monkeystore.p12  
server.ssl.key-store-password=motdepasse  
server.ssl.key-store-type=PKCS12  
server.ssl.key-alias=monalias
```

7.2 Résultat

- L'application est accessible en **HTTPS**
- Communication sécurisée via SSL/TLS

8. Cas d'utilisation : Application Desktop de Signature Numérique

8.1 Contexte

Une application desktop permet de :

- Signer des documents (PDF, XML...)
- Vérifier les signatures

8.2 Étapes

1. Générer la clé

Avec keytool (voir section 4)

2. Charger la clé dans l'application

```
PrivateKey privateKey = (PrivateKey) keystore.getKey("monalias", password);
```

3. Signer un document

```
Signature signature = Signature.getInstance("SHA256withRSA");  
signature.initSign(privateKey);
```

```
byte[] data = Files.readAllBytes(Paths.get("document.txt"));  
signature.update(data);
```

```
byte[] signatureBytes = signature.sign();
```

4. Vérifier la signature

```
Signature signature = Signature.getInstance("SHA256withRSA");  
signature.initVerify(publicKey);
```

```
signature.update(data);
```

```
boolean isValid = signature.verify(signatureBytes);
```

8.3 Résultat

- Garantie d'intégrité du document
- Authentification de l'auteur
- Non-répudiation

9. Bonnes pratiques

- Utiliser **PKCS12** au lieu de JKS
- Protéger les mots de passe du keystore

- Ne jamais exposer une clé privée
- Utiliser des clés d'au moins **2048 bits**
- Mettre en place une rotation des certificats

10. Conclusion

Le Java Keystore est un outil puissant pour :

- Sécuriser les communications web (HTTPS)
- Gérer les identités numériques
- Implémenter des systèmes de signature et chiffrement